

Data Structures — Հարցազրույցի ամբողջական ուղեցույց

Junior / Mid ծրագրավորողի interview presentation

Այս ուղեցույցը ընդգրկում է բոլոր հիմնական data structure-ները, vor հարցումները interview-ում՝ Array, Linked List, Stack, Queue, Hash Table, Tree, BST, Heap, Graph, Trie: ամեն մեկի **ինչ** **e**, **ներքին** **կարգավիճակ**, **գործողությունների** **complexity**, **C++** **կոդ/STL**, **երբ** **օգտագործել**, **և** **interview** **հարցեր**: **Verjume** **hamematakan** **aghyusak** **ev** **cheat sheet**.

0. Ինչ է Data Structure-y

Data structure-y tvyalneri kazmakerpman u pahpanman dzev e, vor tuyl e talis ardyunavet mutq, poxum, u vorosum. Chisht DS-i entrutyuny voroshum e algoritmi aragutyuny.

Ընտրության խափանիչներ:

- Ինչ գործողություններ (access / search / insert / delete)
- Վրոնք են հաճախակի
- Complexity (time / space)
- Memory (contiguous te scattered)

Երկու մեծ խումբ.

Linear: Array, Linked List, Stack, Queue

Non-linear: Tree, BST, Heap, Graph, Trie, Hash Table

1. Array

Խնդիր

Nuyn tipi element-neri **contiguous** (koxq-koxqi) havaqacu, index-ov mutq.

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[2];    // 30 -> O(1) unix mutq  
  
vector<int> v = {10, 20, 30};    // dynamic array (STL)
```

Complexity

Access (index)	$O(1)$	<- amenamec aravelutyuny
Search (unsorted)	$O(n)$	
Insert (verj)	$O(1)^*$	(vector amortized)
Insert (mej)	$O(n)$	(element-ner sharzum)
Delete (mej)	$O(n)$	

Erb ogtagorcel

Random access petq e, chapy hayni e, iteration/sorting/searching. **DEFAULT** entrutyun.

2. Linked List

Ինչ է

Node-neri sharg, amen node nıui `data` + `pointer(ner)` hajord (u nax) node-in. Memory-um **scattered**.

```
struct Node {  
    int data;  
    Node* next;  
};
```

Singly: [10|*]->[20|*]->[30|null]

Doubly: null<-[10]<->[20]<->[30]->null

Complexity

Access (index)	$O(n)$	<- index chka, qayl petq
Search	$O(n)$	
Insert (head)	$O(1)$	
Insert/Delete	$O(1)$	(ete iterator/pointer unes)

Reverse Linked List (klasik interview)

```
Node* reverse(Node* head) {  
    Node* prev = nullptr;  
    while (head) {  
        Node* next = head->next;  
        head->next = prev;  
        prev = head;  
        head = next;  
    }  
    return prev;  
}
```

Erb ogtagorcel

Hachaxaki insert/delete mejtexum, chapy anhayt e, iterator stability petq e.

STL `std::list` (`doubly`), `std::forward_list` (singly).

3. Stack (LIFO)

Ինչ է

Last In First Out. element-ner avelanum/hanvum en miayn mek codic (top).

```
stack<int> st;
st.push(10);    // avelacnum very
st.push(20);
st.top();       // 20 (verjin avelacracy)
st.pop();       // hanum very
st.empty();
```

```
push 10,20,30:  |30| <- top
                  |20|
                  |10|

pop -> 30
```

Complexity

```
push  O(1) | pop O(1) | top O(1)
```

Երբ օգտագործել

Function call stack, undo/redo, expression evaluation, DFS, balanced parentheses, backtracking.

```
// Balanced parentheses
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c=='(') st.push(c);
        else if (c==')') { if (st.empty()) return false; st.pop(); }
    }
    return st.empty();
}
```

4. Queue (FIFO)

Ինչ է

First In First Out. avelanum e verjic (back), hanvum e sksic (front).

```
queue<int> q;  
q.push(10);           // back-um  
q.push(20);  
q.front();            // 10  (araji avelacracy)  
q.pop();              // hanum front-ic
```

```
push 10,20,30:  front[10][20][30]back  
pop -> 10
```

Complexity

```
push 0(1) | pop 0(1) | front 0(1)
```

Tesaknery

```
queue           -> sovorakan FIFO  
deque           -> erku codic push/pop (0(1))  
priority_queue  -> amenamec/pokry vercnum (heap)
```

Erb ogtagorcel

BFS, task scheduling, buffering, producer-consumer.

5. Hash Table

Ինչ է

Key -> hash function -> bucket index. Mijinum $O(1)$ mutq.

```
unordered_map<string,int> m;  
m["age"] = 25;           // 0(1) average  
m.count("age");         // goyutyun  
  
unordered_set<int> s;  
s.insert(5);
```

```
hash(key) -> bucket  
Bucket 3: (10)->(20)    <- collision (chaining)
```

Complexity

```
insert/find/erase: 0(1) average, 0(n) worst (collision/rehash)
```

Էրբ ogtagorcel

Frequency count, duplicate detection, caching, fast lookup (kargy petq che).

6. Tree & Binary Tree

ኮህኒ ቲ

Hierarchical structure. root + child-ner. Binary tree` amen node ≤ 2 child.

```
struct TreeNode {  
    int val;  
    TreeNode* left;  
    TreeNode* right;  
};
```

```
    1  
  / \  
 2   3  
 / \  
4   5
```

Traversal-ner (klasik interview)

```
void inorder(TreeNode* r) {    // Left, Root, Right  
    if (!r) return;  
    inorder(r->left);  
    cout << r->val << " ";  
    inorder(r->right);  
}  
  
void preorder(TreeNode* r) {    // Root, Left, Right  
    if (!r) return;  
    cout << r->val << " ";  
    preorder(r->left); preorder(r->right);  
}  
  
void postorder(TreeNode* r) {    // Left, Right, Root  
    if (!r) return;  
    postorder(r->left); postorder(r->right);  
    cout << r->val << " ";  
}  
  
// Level-order (BFS) -> queue-ov
```

Terminology

Root -> ver | Leaf -> child chunecogh | Height -> longest root->leaf
Depth -> root-ic hasce | Subtree -> node + ir zavaḱner

7. Binary Search Tree (BST)

Ինչ է

Binary tree, vortex. **left** < **root** < **right** (amen node-i hamar).



Search / Insert

```
TreeNode* search(TreeNode* r, int key) {  
    if (!r || r->val == key) return r;  
    return key < r->val ? search(r->left, key) : search(r->right, key);  
}
```

Complexity

Balanced: search/insert/delete $O(\log n)$
Unbalanced: $O(n)$ <- (sorted insert -> linked list)

Karevor

Inorder traversal BST-i talis e **sorted** kargov. Self-balancing tesaknery. AVL, Red-Black Tree
(`std::map` / `std::set` -> Red-Black Tree).

8. Heap (Priority Queue)

ኮህህ ነ

Complete binary tree. **Max-heap**. parent $\geq$ child; **Min-heap**. parent $\leq$ child. Root-y misht max/min.

Max-heap:

```
      100
     /  \
    19   36
   /  \
  17   3
```

```
priority_queue<int> maxHeap;           // default max
maxHeap.push(3); maxHeap.push(10);
maxHeap.top();    // 10 (amenamecy)

priority_queue<int, vector<int>, greater<int>> minHeap; // min
```

Complexity

```
push (insert)  O(log n)
pop (extract)  O(log n)
top (peek)     O(1)
build heap     O(n)
```

ኤርብ ዓገታዎርረል

Top-K elements, Dijkstra, scheduling, median finding, heap sort.

9. Graph

hũz e

Vertices (V) + edges (E). Directed/undirected, weighted/unweighted.

```
// Adjacency list (amenataracvac)
vector<vector<int>> adj(n);
adj[u].push_back(v);           // undirected -> erku uxxutyamb

// Adjacency matrix
vector<vector<int>> mat(n, vector<int>(n, 0));
mat[u][v] = 1;
```

Traversal

BFS -> queue, level-by-level, shortest path (unweighted)
DFS -> stack/recursion, cycle/topo/components
Erkusn el $O(V + E)$

Complexity (representation)

Adjacency List: space $O(V+E)$, edge check $O(\text{degree})$ <- sparse
Adjacency Matrix: space $O(V^2)$, edge check $O(1)$ <- dense

Erb ogtagorcel

Networks, maps, dependencies, social graphs, shortest path.

10. Trie (Prefix Tree)

hüz e

Tree string-eri hamar, amen node` mek char. Prefix-eri arag voronum.

```
      root
     /   |
    c    b
    |    |
    a    a
    |    |
    t    t    -> "cat", "bat"
```

Complexity

insert / search / prefix: $O(L)$ (L = baři erkarutyun)

Erb ogtagorcel

Autocomplete, spell checker, prefix search, dictionary, IP routing.

11. Համեմատական աղյուսակ

Structure	Access	Search	Insert	Delete	Memory
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	contiguous
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	scattered
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	LIFO
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	FIFO
Hash Table	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	buckets
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	tree
Heap	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$	tree/array
Trie	-	$O(L)$	$O(L)$	$O(L)$	tree

* iterator/pointer-i ահայտյան.

12. Erb vory ogtagorcel (voroshman ughecuyc)

Random access, index	-> Array / vector
Hachaxaki insert/delete mejtexum	-> Linked List
LIFO (undo, DFS, parentheses)	-> Stack
FIFO (BFS, scheduling)	-> Queue
Fast lookup, frequency, dedup	-> Hash Table (unordered_map/set)
Sorted data + range query	-> BST (map/set)
Top-K / min-max / priority	-> Heap (priority_queue)
Prefix / autocomplete	-> Trie
Networks / paths / dependencies	-> Graph

13. Interview հարցեր & պատասխաններ

Array vs Linked List?

Array *contiguous*, $O(1)$ access, $O(n)$ insert/delete *mejtexum*, *cache-friendly*. Linked List *scattered*, $O(n)$ access, $O(1)$ insert/delete (*iterator-ov*), *aveli shat memory*.

Stack vs Queue?

Stack *LIFO* (*verjin avelacracy araji hanvum*); Queue *FIFO* (*araji avelacracy araji hanvum*).

Erb hash table, erb BST?

Hash table (*unordered_map*) $O(1)$ avg lookup, *kargy petq che*. BST (*map*) $O(\log n)$, sorted, range query / lower_bound petq e.

BST inorder traversal inch e talis?

Sorted *karg* (*left < root < right*).

Inchu BST-y karo e $O(n)$ darnal?

Ete unbalanced (sorted insert) -> darnum e linked list-i pes. Lucum` AVL / Red-Black Tree.

Max-heap vs min-heap?

Max-heap *root amenamecy* (*parent >= child*); min-heap *root amenapokry*.

BFS vs DFS?

BFS *queue*, *level-by-level*, *shortest path unweighted*; DFS *stack/recursion*, *cycle/topo/components*.

Adjacency list vs matrix?

List $O(V+E)$ space, sparse graph-i hamar; matrix $O(V^2)$, edge check $O(1)$, dense-i hamar.

Trie inchi hamar e?

Prefix search, autocomplete, dictionary. $O(L)$ operations.

Priority queue inchi e ners?

Heap (default max-heap C++-um). push/pop $O(\log n)$, top $O(1)$.

Cheat Sheet

LINEAR:

Array -> $O(1)$ access, contiguous, DEFAULT
Linked List -> $O(1)$ insert/delete, $O(n)$ access, scattered
Stack (LIFO)-> push/pop/top $O(1)$ | DFS, undo, parentheses
Queue (FIFO)-> push/pop/front $O(1)$ | BFS, scheduling

NON-LINEAR:

Hash Table -> $O(1)$ avg lookup, unsorted | frequency, dedup
BST -> $O(\log n)$ balanced, sorted, inorder=sorted | map/set
Heap -> top $O(1)$, push/pop $O(\log n)$ | priority_queue, top-K
Trie -> $O(L)$ prefix ops | autocomplete
Graph -> BFS(queue)/DFS(stack) $O(V+E)$ | adj list (sparse) / matrix (dense)

TRAVERSALS (tree):

inorder = Left Root Right (BST -> sorted)
preorder = Root Left Right
postorder = Left Right Root
level = BFS (queue)

CHOOSE:

index -> array | insert/delete -> list | LIFO -> stack | FIFO -> queue
lookup -> hash | sorted -> BST | priority -> heap | prefix -> trie